

CYBERSECURITY INTERNSHIP

Name: Hammad Rasheed

ID: DHC-5590

Week 5

Ethical Hacking & Exploiting Vulnerabilities

Task 1:

Reconnaissance

Scan Target

```
nmap localhost
```

or

```
nmap <target-ip>
```

Document Open Ports

Example:

Port	Service
80	HTTP
22	SSH

```
hammad@Kali: ~  
Session Actions Edit View Help  
hammad@Kali: ~ hammad@Kali: ~  
  
(hammad@Kali)-[~]  
$ nmap 192.168.77.7  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-09 01:10 +0500  
Nmap scan report for 192.168.77.7  
Host is up (0.00016s latency).  
Not shown: 999 filtered tcp ports (no-response)  
PORT      STATE SERVICE  
80/tcp    open  http  
  
Nmap done: 1 IP address (1 host up) scanned in 5.32 seconds  
  
(hammad@Kali)-[~]  
$
```

Task 2

SQL Injection Testing

Step 1: Test Application

Run SQLMap against your own test application:

```
sqlmap -u "http://localhost:3000/login?id=1"
```

Document whether vulnerabilities are found.

```
Kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help
hammad@Kali: ~
zsh: corrupt history file /home/hammad/.zsh_history
(hammad@Kali)~$ sqlmap -u 'http://192.168.77.7:3000/#/login'
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
[*] starting @ 01:18:04 /2026-06-09/

[01:18:05] [INFO] testing connection to the target URL
[01:18:05] [INFO] checking if the target is protected by some kind of WAF/IPS
[01:18:06] [INFO] testing if the target URL content is stable
[01:18:06] [INFO] target URL content is stable
[01:18:06] [CRITICAL] no parameter(s) found for testing in the provided data (e.g. GET parameter 'id' in 'www.site.com/index.php?id=1'). You are advised to rerun with '--crawl=2'

[*] ending @ 01:18:06 /2026-06-09/
```

Step 2: Fix SQL Injection

Bad:

```
const query =
"SELECT * FROM users WHERE id="+id;
```

Good:

```
db.query(
  "SELECT * FROM users WHERE id=?",
  [id]
);
```

Prepared statements prevent SQL injection.

Task 3: CSRF Protection

Step 1: Install

```
npm install csurf
```

Step 2: Configure

```
const csrf = require("csurf");
```

```
app.use(csrf());
```

Step 3: Add Token

```
<input type="hidden"  
name="_csrf"  
value="<%= csrfToken %>">
```

Step 4: Test With Burp Suite

1. Open Burp Suite
2. Intercept request
3. Remove CSRF token
4. Send request

Expected:

403 Forbidden

This proves CSRF protection works.
